

Tema 4.1. Composición

1. En C, podemos crear estructuras mayores **componiendo** unas con otras, que suelen describirse como "A tiene-un/tiene-varios B". Pon un ejemplo, empleando `struct`, de una línea de puntos, donde puntos tienen dos coordenadas (x e y), y la línea esta hecha de dos puntos. Incluye una función

para calcular la distancia entre puntos y otra para hallar la longitud de una línea.

```
#include <stdio.h>
#include <math.h>

/* Un punto en 2D */
struct Punto {
    double x;
    double y;
};

/* Una línea formada por dos puntos */
struct Linea {
    struct Punto inicio;
    struct Punto fin;
};

/* Calcula la distancia entre dos puntos */
double distancia(struct Punto a, struct Punto b) {
    double dx = b.x - a.x;
    double dy = b.y - a.y;

    return sqrt(dx * dx + dy * dy);
}

/* Calcula la longitud de una línea */
double longitudLinea(struct Linea l) {
    return distancia(l.inicio, l.fin);
}

int main() {
    struct Punto p1 = {0.0, 0.0};
    struct Punto p2 = {3.0, 4.0};

    struct Linea linea = {p1, p2};

    printf("Distancia entre puntos: %.2f\n", distancia(p1, p2));
    printf("Longitud de la línea: %.2f\n", longitudLinea(linea));

    return 0;
}
```

```
}
```

Salida de este programa:

Distancia entre puntos: 5.00

Longitud de la línea: 5.00

(*)-> ¿Donde ocurre la composición?

```
struct Linea {  
    struct Punto inicio;  
    struct Punto fin;  
};
```

2. Ahora transforma ese ejemplo a orientación a objetos con Java, para tener un primer ejemplo de **composición en orientación a objetos. Crea una clase `Punto`, y una clase `Linea`. La clase `Punto` debe tener un método para calcular distancia a otro `Punto` y `Linea` debe tener un método para calcular su longitud. Gracias a la ocultación de información, supera a C, garantizando que los puntos sean inmutables, al igual que la línea, que una vez creada, no queremos que se modifique de qué a qué puntos va dicha línea.**

En Java podemos expresar la misma idea usando clases y composición:

Un objeto `Linea` tiene dos objetos `Punto`.

Un `Punto` conoce cómo calcular la distancia a otro punto.

Una `Linea` conoce cómo calcular su longitud.

Además, gracias a la ocultación de información (`private`) y al uso de `final`, podemos hacer que los objetos sean inmutables: una vez creados, no pueden cambiar.

```

public class Main {

    public static void main(String[] args) {

        Punto p1 = new Punto(0, 0);
        Punto p2 = new Punto(3, 4);

        Linea linea = new Linea(p1, p2);

        System.out.println("Distancia entre puntos: "
            + p1.distanciaA(p2));

        System.out.println("Longitud de la linea: "
            + linea.longitud());
    }
}

```

```

/* Punto inmutable */
final class Punto {

    private final double x;
    private final double y;

    public Punto(double x, double y) {
        this.x = x;
        this.y = y;
    }

    public double getX() {
        return x;
    }

    public double getY() {
        return y;
    }

    public double distanciaA(Punto otro) {

        double dx = otro.x - this.x;
        double dy = otro.y - this.y;

        return Math.sqrt(dx * dx + dy * dy);
    }
}

```

```

    }
}

/* Línea inmutable compuesta por dos puntos */
final class Linea {

    private final Punto inicio;
    private final Punto fin;

    public Linea(Punto inicio, Punto fin) {
        this.inicio = inicio;
        this.fin = fin;
    }

    public Punto getInicio() {
        return inicio;
    }

    public Punto getFin() {
        return fin;
    }

    public double longitud() {
        return inicio.distanciaA(fin);
    }
}

```

Qué mejora respecto a C??

En C:

cualquiera puede modificar directamente los campos de una estructura;
no hay protección real contra cambios accidentales.

```

private final double x;
private final double y;

```

significa:

private → nadie desde fuera puede modificar los atributos;

final → solo pueden asignarse una vez, en el constructor.

Así, un Punto es realmente inmutable.

Lo mismo ocurre con Línea:

```
private final Punto inicio;  
private final Punto fin;
```

Una vez creada la línea, no puede cambiar qué puntos conecta.

En Java:

3. ¿Qué significa la **multiplicidad** en la composición? En el ejemplo anterior, ¿cuál es la multiplicidad entre Línea y Punto ? Indícalo expresando la multiplicidad en ambas direcciones, de Línea a Punto y de Punto a Línea .

1-¿Qué significa la **multiplicidad** en la composición?

La multiplicidad indica cuántos objetos de una clase pueden estar relacionados con objetos de otra clase.

En composición (y en asociaciones en general), la multiplicidad se expresa en ambos sentidos:

cuántos objetos B puede tener un objeto A;

y con cuántos objetos A puede estar relacionado un objeto B.

```
class Línea {  
    private final Punto inicio;  
    private final Punto fin;  
}
```

una Línea está compuesta por dos Punto.

2.¿Cuál es la multiplicidad entre Línea y Punto ?

(*)->Multiplicidad de Línea a Punto

Una Línea tiene exactamente:

1 punto de inicio

1 punto de fin

En total:

Línea ---- 2 Punto

o más formalmente:

Línea 1 ---- 2 Punto

Es decir:

Cada Línea está relacionada con exactamente 2 objetos Punto.

(*)-> Multiplicidad de Punto a Línea

Un Punto puede pertenecer a:

ninguna línea,

una línea,

o muchas líneas.

Por ejemplo, el punto (0,0) podría usarse en cientos de líneas distintas.

Entonces:

Punto ---- 0..* Línea

***CLASE

En el ejemplo:

-1 Línea se relaciona como mínimo con 2 Puntos y como máximo con 2 Puntos

-1 Punto se relaciona como mínimo con 0 Líneas y como máximo con muchas Líneas

4. ¿Qué significa composición **fuerte** y composición **débil**? ¿Qué consecuencia implica en relación al ciclo de vida de los objetos? Indica a cuál solemos referirnos como "**asociación o agregación**" y a cuál como "**composición**" propiamente.

1-¿Qué significa composición **fuerte** y composición **débil**? (agregación / asociación)

COMPOSICION DEBIL_____

En una composición débil, el objeto contenido:

- puede existir independientemente del contenedor;
- puede compartirse entre varios objetos;
- no "muere" cuando desaparece el objeto que lo usa.

¿Qué consecuencia implica en relación al ciclo de vida de los objetos?

Los ciclos de vida son independientes:

COMPOSICION FUERTE_____

En una composicion fuerte, el objeto conetenido:

- no tiene sentido fuera del objeto contenedor;
- pertenece exclusivamente a él;
- su ciclo de vida depende del contenedor.

Esto es lo que normalmente se llama simplemente:

- composición.

¿Qué consecuencia implica en relación al ciclo de vida de los objetos?

Los ciclos de vida están ligados:

El objeto "parte" nace y muere con el "todo".

***CLASE

Composición fuerte vs debil

->Fuerte: El contenedor (p.ej Linea) es el que crea los objetos que contiene (p.ej Punto) y estos no viven más allá del contenedor

Tu puedes hacer un punto y ser independiente de la linea

->Debil: El contenedor y contenido tienen ciclos de vida independientes (p.ej:Los objetos Punto pueden vivir sin estar en objetos Linea)

UML

-Composicion propiamente dicho-> composición fuerte

-Asociación o agregación -> composición debil

5. Cuando una clase usa a otra al recibirla o devolverla como parámetro en algún método, al hacer new dentro de un método, o al usarlas como variables locales, ¿hablamos de composición o de "dependencia"?

En esos casos hablamos de dependencia, no de composición.

La idea es: una clase "usa" temporalmente a otra

,pero:

no la posee como parte de su estado interno;

no forma parte permanente del objeto

COMPOSICION VS DEPENDENCIA

(1)-> Composición

Hay composición cuando un objeto contiene a otro como atributo:

```
class Linea {  
    private Punto inicio;  
    private Punto fin;  
}
```

Aquí una Linea tiene puntos.

Los puntos forman parte del estado de la línea.

(2)-> Dependencia

Hay dependencia cuando una clase simplemente:

-Usa otra clase en un método;

-Crea objetos temporalmente;

-Trabaja con variables locales.

Casos típicos:

-Recibir un objeto por parametro

```
```java
```

```
class Impresora {

 public void imprimir(Punto p) {
 System.out.println(p.getX());
 }
}
```

Impresora depende de Punto porque usa esa clase.

Pero NO la contiene.

-Devolver un objeto

```
java public Punto crearOrigen() { return new Punto(0, 0); }
```

También es dependencia.

-Hacer un new dentro de un metodo

```
java public void ejemplo() { Punto p = new Punto(1, 2); }
```

La clase depende de Punto.

-Variables locales

```
public void mover() {
 Punto temporal = new Punto(3, 4);
}
```

Sigue siendo dependencia.

EN RESUMEN:

Si aparece como atributo:

```
private Punto p;
```

normalmente hablamos de:

composición / agregación

Si solo aparece dentro de métodos:

```
Punto p = ...
```

o como parámetro:

```
metodo(Punto p)
```

entonces hablamos de:

dependencia

\*\*\*CLASE

-> Ejemplos de "dependencia", NO de Composición

ej:Punto depende de String y StringBuilsder...

```
class Punto{
 public String to String (){
 StringBuilder sb=new String Builder();
 }
}
```

```
class OperadorFichero{
 public static String leerFichero(Path p);
}
```

**6. En el ejemplo anterior de línea y punto, programa la relación entre Línea y Punto de dos formas. Una como composición fuerte, donde el ciclo de vida de los puntos está ligado al de Línea y otra como composición débil, donde no.**

(1)-> COMPOSICIÓN FUERTE

Aquí los Punto pertenecen exclusivamente a Línea.

-La línea crea los puntos.

-Desde fuera no se accede directamente a ellos.

-Si desaparece la línea, también desaparecen sus puntos.

```
public class Main {

 public static void main(String[] args) {

 Linea linea = new Linea(0, 0, 3, 4);

 System.out.println("Longitud: " + linea.longitud());
 }
}
```

```
/* Punto inmutable */
final class Punto {

 private final double x;
 private final double y;

 public Punto(double x, double y) {
 this.x = x;
 this.y = y;
 }

 public double distanciaA(Punto otro) {

 double dx = otro.x - this.x;
 double dy = otro.y - this.y;

 return Math.sqrt(dx * dx + dy * dy);
 }
}
```

```
/* Composición fuerte */
final class Linea {

 private final Punto inicio;
 private final Punto fin;

 public Linea(double x1, double y1,
 double x2, double y2) {

 this.inicio = new Punto(x1, y1);
 this.fin = new Punto(x2, y2);
 }
}
```

```
}

 public double longitud() {
 return inicio.distanciaA(fin);
 }
}
```

¿Por qué es composición fuerte?

Porque:

Los Punto nacen dentro de Linea y nadie más los comparte.

La línea “posee” completamente sus puntos.

(2)-> COMPOSICIÓN DÉBIL

Composición débil (agregación)

Aquí los puntos existen independientemente.

-Los puntos se crean fuera.

-Varias líneas podrían compartir el mismo punto.

-Los puntos pueden seguir existiendo aunque desaparezca la línea.

```

public class Main {

 public static void main(String[] args) {

 Punto p1 = new Punto(0, 0);
 Punto p2 = new Punto(3, 4);

 Linea linea = new Linea(p1, p2);

 System.out.println("Longitud: " + linea.longitud());
 }
}

```

```

/* Punto immutable */
final class Punto {

 private final double x;
 private final double y;

 public Punto(double x, double y) {
 this.x = x;
 this.y = y;
 }

 public double distanciaA(Punto otro) {

 double dx = otro.x - this.x;
 double dy = otro.y - this.y;

 return Math.sqrt(dx * dx + dy * dy);
 }
}

```

```

/* Composición débil / agregación */
final class Linea {

 private final Punto inicio;
 private final Punto fin;

 public Linea(Punto inicio, Punto fin) {
 this.inicio = inicio;
 }
}

```

```

 this.fin = fin;
 }

 public double longitud() {
 return inicio.distanciaA(fin);
 }
}

```

¿Por qué es composición débil?

Porque:

los Punto existen antes que la Linea podrían seguir existiendo después.

Incluso podrían compartirse:

```

Punto origen = new Punto(0,0);

Linea l1 = new Linea(origen, new Punto(1,1));
Linea l2 = new Linea(origen, new Punto(2,2));

```

\*\*\*CLASE

COMPOSICIÓN FUERTE

```

class Linea{
 private Punto p1;
 private Punto p2;

 public Linea(double x1, double y1, double x2, double y2) {
 this.p1=new Punto (x1,y1);
 this.p2=new Punto (x2,y2);
 }
}

```

DIFERENCIA ESENCIAL

Composición fuerte: Linea crea y posee los Punto

Composición débil: Linea solo referencia Punto ya existentes

## 7. En Java, en la composición fuerte, ¿cuando el contenedor destruye los objetos? No se observa que Linea destruya los Punto explícitamente, ¿Por qué?

Porque en Java normalmente no destruimos objetos explícitamente.

Eso lo hace automáticamente el garbage collector (GC).

La clave es entender que:

composición fuerte NO significa "destrucción manual"

sino:

"dependencia de existencia"

En Java

Java usa memoria automática.

Los objetos se eliminan cuando: ya no existe ninguna referencia accesible hacia ellos

Entonces el recolector de basura los libera.

Qué ocurre en la composición fuerte

Mira este ejemplo:

```
Linea linea = new Linea(0,0,3,4);
```

```
private final Punto inicio;
```

```
private final Punto fin;
```

Los únicos objetos que conocen esos puntos son:

la propia Linea

Entonces:

la Linea queda inaccesible y también sus Punto.

El GC detecta eso y elimina los tres objetos.

¿Quién destruye los puntos?

No los destruye explícitamente Linea.

Lo que ocurre es:

al desaparecer Linea, desaparecen las únicas referencias a Punto y por tanto los puntos también quedan inaccesibles.

Por eso hablamos de ciclo de vida ligado

Porque:

la existencia de Punto depende de Linea

No porque haya código:

```
destroy(punto);
```

(Java no funciona así.)

En composición débil

En cambio:

```
Punto p1 = new Punto(0,0);
```

```
Linea l = new Linea(p1, otro);
```

```
l = null;
```

Aquí:

p1 sigue existiendo

porque aún hay una referencia:

p1

Entonces el GC NO lo elimina.

RESUMEN:

En Java, “destruir” realmente significa:

quedarse sin referencias alcanzables

Así que:

Composición fuerte

el contenedor posee las únicas referencias importantes

Composición débil

otros objetos también pueden seguir referenciando la parte

\*\*\*CLASE

La vida de punto termina cuando es inaccesible y en el ejemplo ocurre cuando linea deja de serlo a su vez

Por tanto, cuando linea "es basura" , también lo serán sus puntos y serán eliminados de memoria por el recolector de basura

**8. Pon un ejemplo de composicion débil entre un departamento que tiene varios profesores.**

**Implementa dos composiciones a la vez: entre el departamento y todos sus profesores y entre el departamento y su director, que es un profesor del departamento. Siempre debe haber un director en el departamento desde el inicio. Lanza excepciones si se viola la invariante. Emplea arrays primitivos de Java, estilo `Profesor[]` , con máximo 50, pero no rompas la encapsulación, no desveles que estás empleando un array, permite añadir un `Profesor` al final de la lista, y eliminar un profesor dada su posición. Da acceso a los profesores con un método para saber cuántos hay y otro para obtener un profesor por posición. El director se puede cambiar por otro profesor del departamento. Sin embargo, ten en cuenta esta invariante de clase: el director debe formar siempre parte de la lista de profesores, es**

**decir, ten cuidado al cambiar el director o al eliminar un profesor.**

```
public class Main {

 public static void main(String[] args) {

 Profesor p1 = new Profesor("Ana");
 Profesor p2 = new Profesor("Luis");
 Profesor p3 = new Profesor("Marta");

 Departamento d = new Departamento("Informática", p1);

 d.agregarProfesor(p2);
 d.agregarProfesor(p3);

 System.out.println("Director: "
 + d.getDirector().getNombre());

 System.out.println("Profesores:");

 for (int i = 0; i < d.getNumeroProfesores(); i++) {
 System.out.println("- "
 + d.getProfesor(i).getNombre());
 }

 d.setDirector(p2);

 System.out.println("Nuevo director: "
 + d.getDirector().getNombre());

 d.eliminarProfesor(2);

 System.out.println("Profesores tras borrar uno:");

 for (int i = 0; i < d.getNumeroProfesores(); i++) {
 System.out.println("- "
 + d.getProfesor(i).getNombre());
 }
 }
}
```

```
/* Profesor immutable */
final class Profesor {

 private final String nombre;

 public Profesor(String nombre) {

 if (nombre == null || nombre.isBlank()) {
 throw new IllegalArgumentException(
 "Nombre inválido");
 }

 this.nombre = nombre;
 }

 public String getNombre() {
 return nombre;
 }
}
```

```
/* Departamento */
final class Departamento {

 private static final int MAX_PROFESORES = 50;

 private final String nombre;

 private final Profesor[] profesores;

 private int numProfesores;

 private Profesor director;

 public Departamento(String nombre,
 Profesor director) {

 if (nombre == null || nombre.isBlank()) {
 throw new IllegalArgumentException(
 "Nombre inválido");
 }
 }
}
```

```

 if (director == null) {
 throw new IllegalArgumentException(
 "Debe existir director");
 }

 this.nombre = nombre;

 this.profesores =
 new Profesor[MAX_PROFESORES];

 this.profesores[0] = director;
 this.numProfesores = 1;

 this.director = director;
 }

 public String getNombre() {
 return nombre;
 }

 public Profesor getDirector() {
 return director;
 }

 public void setDirector(Profesor nuevoDirector) {

 if (nuevoDirector == null) {
 throw new IllegalArgumentException(
 "Director nulo");
 }

 if (!contieneProfesor(nuevoDirector)) {
 throw new IllegalArgumentException(
 "El director debe pertenecer "
 + "al departamento");
 }

 this.director = nuevoDirector;
 }

```

```

public void agregarProfesor(Profesor profesor) {

 if (profesor == null) {
 throw new IllegalArgumentException(
 "Profesor nulo");
 }

 if (numProfesores >= MAX_PROFESORES) {
 throw new IllegalStateException(
 "Departamento lleno");
 }

 profesores[numProfesores] = profesor;
 numProfesores++;
}

```

```

public void eliminarProfesor(int posicion) {

 comprobarPosicion(posicion);

 Profesor profesorAEliminar =
 profesores[posicion];

 if (profesorAEliminar == director) {
 throw new IllegalStateException(
 "No se puede eliminar "
 + "al director");
 }

 for (int i = posicion;
 i < numProfesores - 1;
 i++) {

 profesores[i] = profesores[i + 1];
 }

 profesores[numProfesores - 1] = null;

 numProfesores--;
}

```

```

public int getNumeroProfesores() {
 return numProfesores;
}

public Profesor getProfesor(int posicion) {

 comprobarPosicion(posicion);

 return profesores[posicion];
}

private void comprobarPosicion(int posicion) {

 if (posicion < 0
 || posicion >= numProfesores) {

 throw new IndexOutOfBoundsException(
 "Posición inválida");
 }
}

private boolean contieneProfesor(
 Profesor profesor) {

 for (int i = 0; i < numProfesores; i++) {

 if (profesores[i] == profesor) {
 return true;
 }
 }

 return false;
}
}

```

ASPECTOS IMPORTANTES DE LA IMPLEMENTACION:

(\*)-> Encapsulación

Aunque internamente usamos: `Profesor[]`;  
desde fuera NO se ve el array.

El acceso se hace mediante:

`getNumeroProfesores()`  
`getProfesor(int posicion)`

Así podemos cambiar la implementación interna en el futuro sin romper código cliente.

(\*)-> Invariante protegida

-`setDirector()` verifica que el profesor pertenezca al departamento;  
-`eliminarProfesor()` impide borrar al director.

(\*)-> Multiplicidades

Departamento → Profesores

1 → 1..50

Todo departamento tiene entre 1 y 50 profesores.

Departamento → Director

1 → 1

Siempre existe exactamente un director.

Director → Profesores

El director es simultáneamente un profesor del departamento.

\*\*\*CLASE

```

//composicion debil 1 :
//1 Departamento como minimo 0 y como maximo muchos Profesor
//1 Profesor como minomo 0 y como maximo muchos Departamentos
public class Departamento{
 private Profesor[] profesores =new Profesor [50];
 private int numProfesor=0;
}

//composicion 2:
//1 Departamento tiene como minimo 1 y como maximo 1 Profesor director ***
//1 Profesor puede ser director como minimo de 0 y como maximo de muchos Departamentos
private Profesor director;

public Departamento (Profesor director){
 //0.Si director es null lanzamos IAE
 //1.Añadimos el director al conjunto de profesores
 //2.Establecemos ese profesor como director
}

public int getnumProfesores(){ return this.numProfesores; }
public Profesor getProfesor (int pos){
 //0.Validamos pos, y si no valida lanzar IAE
 return this.profesores [pos];
}

public void addProfesor (Profesor p){
 //0. Si ya no hay mas sitio, lanzar IAE o AIOBE (ArrayIndexOutOfBoundsException)
}
public void eliminarProfesor (int pos){
 //0.Si pos no está en el rango correcto (0-numProfesores),
 //lanzar IAE
 //1.Si el profesor en pos ES EL DIRECTOR, lanzar IAE
}

public void cambiarDirector(Profesor nuevoDirector){
 //0.Si nuevoDirector es null, IAE
 //Si nuevoDirector no lo encuentro (bucle de busqueda), lanzo IAE, diciendo que hay que
}

public Profesor getDirector(){
 return this.director;
}

```

```
}
```

- Hay 2 composiciones débiles
- No se expone el array al exterior (imposible garantizar invariante de clase)
- En los métodos que gestionan el departamento se controla que no se viole la invariante de clase

**9. En Java, existen también `List` , cambia y muestra cómo sería el código anterior empleando `List` en vez de arrays primitivos. ¿Qué parte del código original te has ahorrado? Además, fíjate en el método `getProfesor(int pos)` : si en su lugar existiera un método que devolviera todos los profesores a la vez, ¿qué problema tendría devolver directamente la lista interna? ¿Cómo lo resolverías?**

```
import java.util.ArrayList;
import java.util.List;

final class Profesor {

 private final String nombre;

 public Profesor(String nombre) {
 if (nombre == null || nombre.isBlank()) {
 throw new IllegalArgumentException();
 }
 this.nombre = nombre;
 }

 public String getNombre() {
 return nombre;
 }
}
```

Departamento con List

```
import java.util.ArrayList;
import java.util.List;

final class Departamento {

 private final String nombre;

 private final List<Profesor> profesores;

 private Profesor director;

 public Departamento(String nombre, Profesor director) {

 if (nombre == null || nombre.isBlank()) {
 throw new IllegalArgumentException();
 }

 if (director == null) {
 throw new IllegalArgumentException();
 }

 this.nombre = nombre;
 this.profesores = new ArrayList<>();

 this.profesores.add(director);
 this.director = director;
 }

 public int getNumeroProfesores() {
 return profesores.size();
 }

 public Profesor getProfesor(int pos) {
 comprobarPosicion(pos);
 return profesores.get(pos);
 }

 public void addProfesor(Profesor p) {
 if (p == null) throw new IllegalArgumentException();
 profesores.add(p);
 }

 public void eliminarProfesor(int pos) {
```

```

 comprobarPosicion(pos);

 Profesor eliminado = profesores.get(pos);

 if (eliminado == director) {
 throw new IllegalStateException("No se puede eliminar al director");
 }

 profesores.remove(pos);
 }

 public Profesor getDirector() {
 return director;
 }

 public void cambiarDirector(Profesor nuevo) {

 if (nuevo == null) throw new IllegalArgumentException();

 if (!profesores.contains(nuevo)) {
 throw new IllegalArgumentException(
 "El director debe pertenecer al departamento"
);
 }

 director = nuevo;
 }

 private void comprobarPosicion(int pos) {
 if (pos < 0 || pos >= profesores.size()) {
 throw new IndexOutOfBoundsException();
 }
 }
}

```

1-¿Qué te ahorras usando List?

Aquí está lo importante:

Con arrays tenías que hacer tú:

- controlar capacidad máxima (50)
- gestionar contador manual (numProfesores)

- desplazar elementos al eliminar
- comprobar límites manualmente con lógica más compleja

Ejemplo típico que desaparece:

```
for (int i = pos; i < numProfesores - 1; i++) {
 profesores[i] = profesores[i + 1];
}
```

Con List te ahorras:

- gestión de tamaño interno
- redimensionamiento automático
- desplazamientos internos (remove lo hace solo)
- size() sustituye a numProfesores
- código más seguro y legible

2-¿Qué problema tendría devolver directamente la lista interna?

Si haces esto:

```
java public List<Profesor> getProfesores() { return profesores; }
```

Problema grave:

El exterior puede romper tu invariante:

```
departamento.getProfesores().clear();
```

o incluso:

```
departamento.getProfesores().remove(0);
```

Esto podría eliminar al director sin pasar por tus controles.

3-¿Cómo se soluciona?

Opción 1 (recomendada): devolver vista inmodificable

```
import java.util.Collections;

public List<Profesor> getProfesores() {
 return Collections.unmodifiableList(profesores);
}
```

No se puede modificar desde fuera

Mantiene encapsulación

Protege invariantes

Opción 2: devolver copia defensiva

```
public List<Profesor> getProfesores() {
 return new ArrayList<>(profesores);
}
```

El exterior puede modificar su copia pero NO afecta al objeto interno

ES DECIR,

Encapsulación no es solo ocultar atributos, es impedir que el exterior rompa invariantes.

\*\*\*CLASE -> MIRAR -> NO ESTÁ BN

```
//composicion debil 1 (VERSION CON LIST):
//1 Departamento como minimo 0 y como maximo muchos Profesor
//1 Profesor como minimo 0 y como maximo muchos Departamentos
public class Departamento{
 private List<Profesor> profesores =new ArrayList<>();
 private int numProfesor=0;

 //composicion 2:
 //1 Departamento tiene como minimo 1 y como maximo 1 Profesor director ***
 //1 Profesor puede ser director como minimo de 0 y como maximo de muchos Departamentos
 private Profesor director;
}
```

-No cambia la interfaz publica

-Es más facil implementar algunos metodos delegando en metodos List

-Si se devuelve hay que devolver una copia para proteger la invariante de clase

**10. Al igual que ocurre con las excepciones en Java, que pueden encerrar causas (que son excepciones), de forma recursiva, suponen un tipo especial de composiciones, denominadas composiciones recursivas. Pon un ejemplo en Java de una Persona , que sea inmutable, y que tiene una madre, que es otra Persona . Haz un main con un ejemplo de uso con una familia de personas, desde el nieto hasta la abuela. Enumera algún otro ejemplo clásico de composiciones recursivas.**

\*\*\*CLASE

```
final class Persona {

 private final String nombre;
 private final Persona madre;

 public Persona(String nombre, Persona madre) {

 if (nombre == null || nombre.isBlank()) {
 throw new IllegalArgumentException("Nombre inválido");
 }

 this.nombre = nombre;
 this.madre = madre;
 }

 public String getNombre() {
 return nombre;
 }

 public Persona getMadre() {
 return madre;
 }
}
```

Main con familia (nieto → madre → abuela)

```
public class Main {

 public static void main(String[] args) {

 // Abuela (no sabemos madre)
 Persona abuela = new Persona("Abuela Carmen", null);

 // Madre
 Persona madre = new Persona("Ana", abuela);

 // Nieto
 Persona nieto = new Persona("Luis", madre);

 System.out.println("Nieto: " + nieto.getNombre());
 System.out.println("Madre del nieto: " + nieto.getMadre().getNombre());
 System.out.println("Abuela del nieto: "
 + nieto.getMadre().getMadre().getNombre());
 }
}
```

Ejemplo clásicos de composiciones recursivas:

- 1-Árboles
- 2-Sistema de archivos
- 3-Estructuras organizativas
- 4-Comentarios en foros
- 5-Excepciones

## 11. ¿Qué son las relaciones de composición "bidireccionales"? ¿Qué habría que hacer para implementar este tipo de relación en el ejemplo de Profesor y Departamento ? -> FALTA // VOLVER MIRAR // SIN ACABAR

Las relaciones de composición bidireccionales son un tipo de relación entre clases (en programación orientada a objetos) donde:

Hay composición: una clase “contiene” a otra (relación fuerte todo–parte).

Es bidireccional: ambas clases se conocen entre sí y pueden acceder una a la otra.

Es una relación donde:

Existe una relación todo–parte (composición o agregación)

Y además ambas clases se referencian mutuamente

Departamento → Profesores

Profesor → Departamento

¿Qué significa esto exactamente?

En una composición:

El objeto “parte” no tiene sentido sin el “todo”.

Si el “todo” se destruye, las “partes” también.

En una relación bidireccional:

No solo el todo conoce a las partes, sino que cada parte también conoce al todo.

Ejemplo: Profesor y Departamento

Supongamos:

Un Departamento tiene varios Profesor.

Cada Profesor pertenece a un único Departamento.

Esto sería una composición bidireccional si:

El Departamento gestiona la vida de los Profesor.

Cada Profesor tiene una referencia a su Departamento.

IMPORTANTE: mantener la coherencia

En relaciones bidireccionales hay que asegurarse de que:

Si añades un Profesor al Departamento → también se actualice su departamento.

Si lo eliminas → se rompa la referencia en ambos lados.

Esto se hace normalmente centralizando la gestión en una clase (en este caso, Departamento).

\*\*\*CLASE

Tanto el departamento conoce los profesores como los profesores conocen el departamento al que pertenecen-> Relacion bidireccional

Si fuese unidireccional no se vería reflejado en el código, se deduce. Pero la bidireccional si se refleja, la cual exige programar cuidadosamente para mantener la consistencia

-> Ej: Si añadido un profesor al departamento, debo actualizar la referencia al Departamento desde Profesor

Para implementarla, la relac